



Prompt Engineering ■ Claude Code

Plan de la formation

Techniques de prompting & cas d'usage Claude Code

Deux temps : piloter le modèle par le prompt, puis l'agent qui agit

La formation en deux temps

Partie 1 — Prompt Engineering

Savoir **piloter le modèle par le prompt** : du prompt naïf aux techniques avancées de raisonnement et de décision.

Exemples Dev & DevOps.

Partie 2 — Claude Code

Passer du prompt à **l'agent qui agit** : installation, briques (commandes, Skills, hooks, MCP, plugins), puis **11 cas d'usage** Dev / DevOps.



Fil rouge : un LLM **prédit du texte plausible** ; la qualité vient du **prompt**, du **contexte** et des **outils**.

Plan de la formation

Partie 1 — Prompt Engineering



Les techniques de prompting présentées

Base & structuration

- **Zero-shot** — réflexe par défaut, sans exemple
- **Few-shot** — imposer une convention par l'exemple
- **R-T-C** — Rôle · Tâche · Contexte (*role prompting*)
- **Balises XML** — séparer consigne & données
- **Format imposé** — diff / JSON / étapes
- **Contexte & contraintes** — anti-hallucination
- **Itération chirurgicale** — corriger sans tout refaire

Raisonnement, décision & production

- **Chain-of-thought** — raisonner par étapes
- **Prompt chaining** — décomposer une grosse tâche
- **Self-Consistency** — décision critique (go/no-go)
- **Tree of Thoughts** — comparer plusieurs options
- **Indice de confiance** — seuil + escalade humaine
- **Active-Prompt** — amélioration continue sur cas réels
- **ReAct** — base des agents (Partie 2)

Socle commun : **anatomie d'un bon prompt** (R-T-C + format + contraintes) et le principe *garbage in, garbage out*. Les techniques se **combinent**.

Plan de la formation

Partie 2 — Claude Code



Les briques de Claude Code



- **Commandes** — raccourcis /nom (intégrées ou personnalisées).
- **Skills** — savoir-faire chargés automatiquement ; **sous-agents** & **multi-agents** pour déléguer/orchestrer.
- **Hooks** — garde-fous déterministes ; **MCP** — outils & données externes ; **Plugins** — empaqueter & partager.

Les 11 cas d'usage Dev / DevOps (UC1–UC11)

1 · Code, collaboration & qualité

UC1 Git & GitHub · UC2 CI/CD · UC3 Tests

2 · Conteneurs & orchestration

UC4 Docker · UC5 Cluster K8s · UC6 Helm

3 · Données & application

UC7 Base de données · UC8 UI / frontend

4 · Performance & fiabilité

UC9 Load test · UC10 Observabilité · UC11
Incident & logs

Chaque cas mobilise les briques (commandes, Skills, sous-agents, hooks, MCP) et alimente le **capstone** end-to-end. **Bonus** : networking, backup/restore, GitOps, FinOps, documentation.

Cas d'usage en détail (1/2) — UC1 à UC6

Format : **UC** — ce qu'on développe ; *apport de Claude*.

UC1 Git & GitHub — PR propre auto ; *commits/descriptions rédigés, risques signalés*.

UC2 CI/CD — pipeline build/test/scan/deploy ; *généré correct en minutes*.

UC3 Tests — intégration (vrais endpoints) ; *cas limites couverts*.

UC4 Docker — images multi-stage non-root ; *plus légères et plus sûres*.

UC5 Cluster K8s — k3d + déploiement (probes/HPA) ; *manifests générés, pods débogués*.

UC6 Helm — chart multi-environnements ; *un seul chart dev/preprod*.

Cas d'usage en détail (2/2) — UC7 à UC11

UC7 Base de données — schéma + migrations + seed ; *requêtes optimisées*.

UC8 UI / frontend — page modernisée + maquette ; *sans expert front*.

UC9 Load test — scénario k6 (500 VUs) ; *goulot identifié avant la prod*.

UC10 Observabilité — Prometheus/Grafana + alertes ; *dashboards générés*.

UC11 Incident & logs — runbook + post-mortem ; *cause racine, MTTR réduit*.

On commence

Du prompt à l'agent qui agit.



Bassem Ben Hamed